

Прелюдия

Для прочтения и осознания данной главы читателю потребуются определённые знания и умения. В частности навык работы с языком C и, отчасти, C++.

Под “инфраструктурой” понимается, как минимум, наличие компилятора с языка высокого уровня и эмулятора процессора, чья архитектура была разобрана в предыдущей главе. C будет тем самым языком, для которого мы напишем компилятор, ну а писать его будем на кондовом C++.

Литература по C (введение¹)

- Классика жанра. [Брайан Керниган, Деннис Ритчи. “Язык программирования Си”](#).
Есть множество изданий:
Москва: Финансы и статистика, 1992. — ISBN 5-279-00473-1
Санкт-Петербург: Невский диалект, 2000. — ISBN 5-7940-0045-7
Москва: Вильямс, 2015. — 304 с. — ISBN 978-5-8459-1975-5
- [Бен Клеменс Язык C в XXI веке](#) ДМК Пресс 2018 (есть издание 2015 г.)
(Ben Klemens 21st Century C O'REILLY 2015) --- ISBN: 978-5-97060-614-8
- [Керниган Б.У., Пайк Р. Практика программирования, 2004.](#)
Питер Ван Дер Линден, «[Expert C Programming: Deep C Secrets](#)» (Prentice Hall, 1994), ISBN 978-0131774292

Раз уж речь зашла о построении компилятора, невозможно пройти мимо классиков

- UNIX programming environment, Brian Kernigan, Rob Pike (Prentice Hall, 1984) --
[ISBN 013-937699-2](#)
UNIX программное окружение, Брайан Керниган, Роб Пайк,
СПб: Символ Плюс, 2003. ISBN 5932860294
- [Aho, Sethi, Ullman, Compilers: Principles, Techniques, and Tools](#), Addison-Wesley, 1986. ISBN 0-201-10088-6
Альфред Ахо, Рави Сети, Джеффри Ульман. Компиляторы. Принципы, технологии, инструменты. Издательство Вильямс, 2001. ISBN 5-8459-0189-8

V1.

Эта версия архитектуры максимально близка к той, что была использована в гл. [5.2 Создаём свою архитектуру](#).

Есть всё же некоторые изменения:

- Опкоды инструкций пронумерованы заново, их можно увидеть в файле [instr.h](#)
- Добавлена инструкция NOP, декодер игнорирует ее при распаковке кода.

Потребность в такой инструкции возникает из-за Vuint7, при упаковке инструкций GOTO и CALL адрес перехода может быть еще неизвестен, так что

¹ в самом деле, что за введение без прелюдии

мы резервируем место под максимальный адрес (в данном случае 2 байта) с помощью инструкций NOP. В дальнейшем, когда адрес перехода определится, запишем его в зарезервированное место и если он окажется короче, то остаток будет проигнорирован декодером.

- Зарезервировано еще несколько инструкций - для вычисления логических выражений (OR, AND, NOT) и битовых операций (BAND, BOR, XOR, BNOR)
- Вместо набора инструкций сравнения (*i eq, i gt, i ne, ...*) использована единая инструкция *i_cmp* с аргументом - типом сравнения. Аргумент это битовая маска из трёх разрядов. Первый (младший) разряд отвечает за “меньше”, средний за “равенство” и третий за “больше”. Так, старая инструкция *i_ge* эквивалентна *cmp b*.

Такой набор инструкций упрощает внутреннее устройство процессора, хотя и ценой распухшего кода, ведь инструкции из однобайтовых стали двухбайтовыми. Впрочем, сейчас нам важнее простота, возможно, в дальнейшем еще вернёмся к этому вопросу.

- Адреса начала стека, начала первого фрейма и стартовая позиция кода теперь не зашиты жестко, а определяются при компиляции/сборке программы

Ассемблер

Задача первой версии ассемблера - перейти от программирования в автокоде к языку ассемблера, где используются мнемоники инструкций вместо опкодов и имена меток вместо абсолютных адресов. Одно это экономит программисту кучу времени в основном из-за того, что избавляет от кучи обидных оценок. Особенно учитывая то, что в нашей архитектуре код упаковывается Vuint7.

Исходный код ассемблера находится по адресу [zasm](#).

Ассемблер написан с использованием генераторов синтаксического (bison) и лексического (flex) анализаторов, что может показаться избыточным для такой, в общем-то, несложной задачи. Но в дальнейшем это сэкономит нам кучу времени и сил.

Использованная грамматика:

```
%start section_list

%%
section_list
    : section
    | section_list section

section
    :      data_section
    |      ccode_section
    |      xcode_section

data_section
```

```

      : ZASM_DATA_SECTION '{' data_list '}'

data_list
  : data_item
  | data_list data_item

data_item
  : ZASM_IDENTIFIER ':' data_elem_list ';'

data_elem_list
  : data_elem
  | data_elem_list data_elem

data_elem
  : ZASM_INTEGER_LITERAL

ccode_section
  : ZASM_CCODE_SECTION
  '{' code_list '}'

code_list
  : code_item
  | code_list ccode_item

code_item
  : ZASM_IDENTIFIER ':'
  | instr_item

xcode_section
  : ZASM_XCODE_SECTION
  '{' code_list '}'

instr_item
  : ZASM_INSTR instr_args ';'
  | ZASM_INSTR ';'

instr_args
  : instr_arg
  | instr_args instr_arg

instr_arg
  : ZASM_FLOAT_LITERAL
  | ZASM_IDENTIFIER
  | ZASM_STRING_LITERAL
  | ZASM_INTEGER_LITERAL

```

Рассмотрим как это выглядит на практике.

Возьмем уже использовавшийся нами пример вычисления факториала:

```

int factorial(int i)
{
    if (i==0) return 1;
    else return i*factorial(i-1);
}
...
factorial(6);

```

На языке ассемблера это будет выглядеть так:

```

DATA {
    a: 0x0001
    b: 0x0002
    c: 0x0003
    d: 0x0004
}

```

Для начала секция глобальных данных. В примере таковые не использовались, но фактически существовали, так что создаем секцию. В ней описаны четыре переменные каждая размером в 32 разряда и определены их значения.

В дальнейшем потребуются секция константных данных, назовем её CDATA.

Секций таких может быть несколько, они могут перемежаться с кодом.

```

CCODE {
@initial:
    exec block1;
    call factorial 4 0;
}
CCODE {
factorial:
    exec block2;
    if @endif1;
    exec block3;
    ret;
@endif1:
    exec block4;
    call factorial 4 0
    exec block5;
    ret;
}

```

Секции типа CCODE содержат код потока управления. В них могут быть инструкции потока управления и именованные метки. На метку можно перейти инструкцией goto или попасть через вызов функции. Если блок не заканчивается инструкцией stop, она добавляется автоматически.

Некоторые метки служебные (@endif1, @initial), поэтому начинаются со специального символа @. Кроме него в имени метки могут быть буквы латинского алфавита, цифры и символ '_'. Символ @ невозможен в идентификаторах языка C, так что такие метки можно использовать в служебных целях без риска пересечься с пользовательскими именами.

```

XCODE {
block1:
    imdpush 6;
    stop;

block2:
    imdpush 1;
    locpush 0;
    eval;
    eq;
    stop;

block3:
    imdpush 1;
    stop;

block4:
    lockpush 0;
    eval;
    imdpush 1;
    sub;
    stop;

block5:
    locpush 0;
    eval;
    mul;
    stop;
}

```

Секция потока исполнения. Их может быть несколько и вполне могут перемежаться с секциями других типов. Содержит набор именованных блоков, по которым к ним обращаются инструкции потока управления.

Собираем zasm, запускаем (`zasm -f test.asm`), на выходе получаем два файла с образами памяти: данных (`data_segment.mem`) и кода (`rom_image.mem`), аналогичных тем, что мы использовали в главе [5.2 Создаём свою архитектуру](#).

Кроме того, ассемблер сообщает нам размер секции глобальных данных - начало стека и первого фрейма, а также начальный адрес для исполнения - в ассемблере ему соответствует метка “@initial”.

```

Initial code address: 0x1a
Stack start position: 0x10

```

Фиг.5.3.1 Адреса в выдаче ассемблера

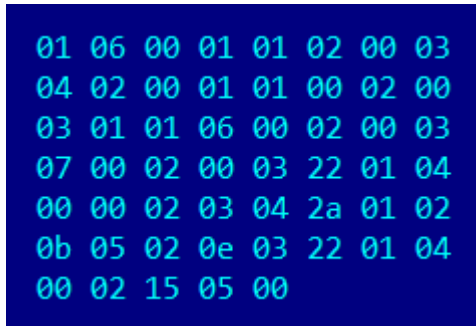
```

00000001 00000002 00000003 00000004

```

Фиг.5.3.3. Содержимое `data_segment.mem`

Сегмент данных - четыре слова с глобальными переменными, далее неинициализированная память под стек.



```
01 06 00 01 01 02 00 03
04 02 00 01 01 00 02 00
03 01 01 06 00 02 00 03
07 00 02 00 03 22 01 04
00 00 02 03 04 2a 01 02
0b 05 02 0e 03 22 01 04
00 02 15 05 00
```

Фиг.5.3.3. Содержимое сегмента кода (rom_image.mem).

Сегмент кода - бинарное представление для нашей архитектуры версии V1.

“Пилоты молчат, смотрят на звёзды.
Задание выполнено ... успешно.
Все улыбаются.” (C)

C

Язык программирования это следующий уровень абстракции, он даёт программисту независимость от конкретной архитектуры процессора и позволяет сосредоточиться на решении задачи в терминах структур данных а не регистров. Язык C был одним из первых языков высокого уровня, предназначенным для системного программирования и единственным, до сих пор, спустя более чем 50 лет активно используемым в этом качестве. Простота, лаконичность, модульность - вот секреты успеха. Компиляторы, операционные системы, базы данных, ... там где нужна высокая производительность - “под капотом” с большой вероятностью вы обнаружите C/C++
А появился C в некотором смысле волею случая.

История возникновения языка.

В 1969 г. компания [Bell Labs](#) вышла из проекта [Multics](#). Multics была новаторским проектом операционной системы, предпринятым совместно Bell Labs, General Electric и Массачусетским Технологическим Институтом (MIT). В проект была заложена целая куча революционных идей:

- вытесняющая многозадачность
- официальное разделение памяти процессов на внутреннюю (оперативную) и внешнюю (файлы)
- централизованная файловая система
- сегментно - страничная виртуальная память
- динамическое связывание - разделяемые библиотеки
- многопроцессорность
- встроенная система безопасности

- конфигурация ОС на-лёту
- в основном написана на языке высокого уровня (PL/1)
- ...

В общем, всё то, чего мы сейчас интуитивно ожидаем от операционной системы, впервые в совокупности появилось именно в Multics.

К сожалению, данный проект, как и многие до и после него, был погребён под собственной сложностью. После выхода Bell Labs из проекта, один из сотрудников, [Кен Томпсон](#) оказался временно не у дел в ожидании назначения, в его распоряжении был свободный компьютер PDP-7.

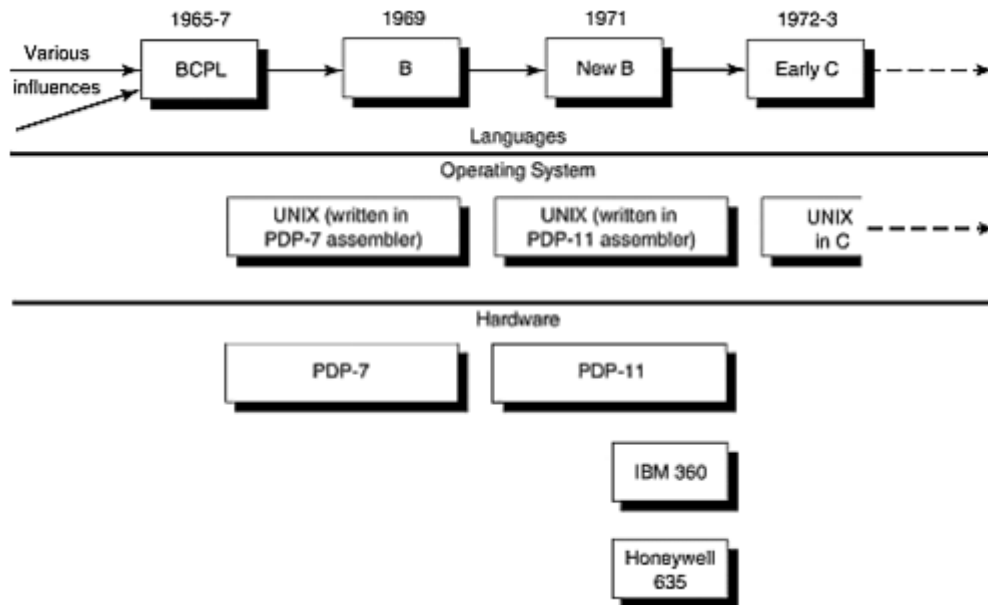
Кен, как человек деятельный, принялся воплощать свои наработки в проекте Multics так, как считал нужным, без бесконечных увязок и согласований. Ядро новой операционной системы было написано на языке ассемблера PDP-7, в новой ОС воплощались основные идеи из Multics, за исключением того, что она была однозадачной. Изначальным названием выбрано Unics² и в нём слышится некоторое подтрунивание над прообразом. Со временем Unics превратилось в UNIX, но это другая история.

Написание ядра операционной системы на ассемблере дело трудоёмкое, приходилось работать со структурами данных, но ассемблер не имеет соответствующих выразительных средств. К тому же вместо PDP-7 Кену Томпсону и его коллеге [Денису Ричи](#) удалось получить доступ к PDP-11. Это определённо послужило стимулом к написанию нового языка программирования высокого уровня.

За основу выбрали [BCPL](#), написанный чуть ранее (1966) Мартином Ричардсом из Кембриджского университета. Рабочим языком Multics был PL/1, который многословен и тяжеловат для системного программирования. Кен Томпсон изрядно упростил его синтаксис, выкинув всё с его точки зрения ненужное, так получился язык, названный **B** (1969). По сравнению с BCPL в **B** нет вложенных процедур, меньше разных циклов, а также использована концепция массивов как указателей со сдвигом, появились операторы ++ и --. Стоит упомянуть, что BCPL в свою очередь является упрощением языка для научных расчетов [CPL](#).

В бестиповом **B** все типы были размером с машинное слово, что было несколько неудобно при написании ядра ОС, кроме того, интерпретируемый язык имел проблемы с производительностью. Поэтому, когда потребовалось перенести работы на PDP-11, Кен Томпсон предпочел написать ядро на ассемблере, а Деннис Ричи портировал **B** в “New B”, который допускал типы разного размера и, самое главное, стал компилируемым. В дальнейшем название “New B” мутировало в “C”.

² *Uniplexed Information and Computing Service*

Фиг.5.3.4 Происхождение C ([отсюда](#))

Язык тем временем развивался и к середине 70-х превратился в тот C, который более привычен современному программисту.

А в 1978 г портируемый компилятор [Стива Джонсона](#) под названием [pcc](#) вышел за пределы Bell Labs и стал весьма популярен. Первый компилятор C от Денниса Ритчи был специфичен к некоторым архитектурным особенностям PDP-11, pcc же использовал [уасс](#) генератор для разбора грамматики и легко мог быть перенесён на любую архитектуру.

В 1978 же году вышла классическая книга Денниса Ритчи и [Брайана Кернигана](#) “[Язык программирования C](#)”, описанный в ней диалект стал стандартом де-факто и дожил до наших дней, известный как “**K&R C**”.

Что отличало (и отличает) C от, например, появившегося одновременно с ним языка [Pascal](#)? Никлаус Вирт создал его после разочаровывающего опыта работы в комитете ALGOL-68. Целью Вирта был инструмент для обучения студентов правильным техникам программирования на языке высокого уровня, что вполне удалось.

C же, по большому счету, не является языком высокого уровня. Язык высокого уровня должен изолировать программиста от особенностей архитектуры, подразумевается что ему присуща [строгая типизация](#), предохраняющая программиста от случайных ошибок при конвертации между типами данных.



Фиг.5.3.5 [Кен Томпсон](#) (справа) и [Деннис Ритчи](#) мучают PDP-11 в Нью-Джерси

При создании C ставилась задача создать инструмент, для решения задач с помощью минимально возможных усилий. Если для этого нужна слабая типизация и преобразования по-умолчанию - то так тому и быть. Ведь значительное количество практикующих программистов лишняя писанина только расстраивает, примерно как длинные слова расстраивают Винни-Пуха.

С точки зрения языка высокого уровня операторы $++/--$ избыточны, ну чего в самом деле стоит написать $p := p + 1$? Неужели компилятор сам не разберётся какой код будет эффективнее? С нынешних представлений - да, разберётся. Но не стоит забывать, что у создателей стояла задача сделать однопроходный транслятор/компилятор потребляющий минимум ресурсов. Да и писанины меньше. Как ни странно, в дальнейшем, уже в контексте C++, операторы $++/--$ очень удачно легли в канву потоковых объектов и итераторов.

И, конечно, отдельное спасибо хочется сказать тому, кто стал использовать скобки `{}` вместо пары BEGIN / END.

Как насчет стандартизации?

Официальный стандарт, одобренный и ANSI и ISO появился в 1990 г..

Он опубликован как ANSI X3.159-1989 (**C89**) и [ISO/IEC 9899:1990](#). К тому времени успело возникнуть немало диалектов, в частности от Microsoft. Этот стандарт

называют **C90** или по привычке **ANSI-C**.

Далее был стандарт ISO/IEC 9899:1990/AMD1:1995 **C95**, где в основном была определена работа с [unicode](#).

за ним последовали

[ISO/IEC 9899:1999](#) известный как **C99** (много всего, в двух словах не передать)
ISO/IEC 9899:2011 ... **C11** (многопоточность, unicode).
ISO/IEC 9899:2018 ... **C17** Уточнённый, исправленный **C11**
ISO/IEC 9899:2024 ... **C23** ...

Текущая поддержка разных стандартов C в GCC доступна [здесь](#).

То же для CLANG [здесь](#).

Про поддержку в MS VS.2019 [здесь](#). В MS VS.2017 - [здесь](#). Не сказать, что Microsoft изо всех сил бьётся за совместимость со стандартом.

Компилятор

За основу возьмём доступную грамматику C11 ([bison](#)(yacc), [flex](#)), не из-за каких-либо особенностей стандарта, а исключительно по причине доступности. Если при построении ассемблера говорилось об избыточности использования генераторов лексического и синтаксического анализаторов, то здесь они очень даже к месту.

На данном этапе нет смысла реализовывать весь язык, сделаем небольшое (но важное) его подмножество - работу с целочисленными переменными, без указателей и массивов. Цель данной работы - продемонстрировать технику компиляции для разработанной ранее стековой архитектуры.

Исходные тексты компилятора доступны [здесь](#), а сейчас рассмотрим основные идеи, заложенные в компилятор.

- 1) Традиционно, компилятор C пишут на C. Когда требуется портировать компилятор на новую платформу, можно взять какой-то очень простой компилятор, тот же рсс, и портировать сначала его. Далее с помощью рсс компилируется нужный компилятор, который в конце концов способен скомпилировать сам себя (bootstrapping). Наша цель - демонстрация возможностей, поэтому там где это экономит время и силы, будет использоваться C++.
- 2) Практически вся память выделяется из т.н. пула памяти (memory-pool). Это механизм, который только выделяет память большими кусками и придерживает до конца. Так можно не заботиться об освобождении памяти, после окончания компиляции пул просто отдаёт всю выделенную память. Такая техника в условиях компиляции, когда аллоцируется большое количество мелких блоков, быстра и экономна.

Ничего нового тут, кстати, нет, большинство трансляторов/компиляторов в том

или ином виде так и поступают.

В нашем случае пул памяти выделяет тегированную память. Каждый аллоцированный кусок предваряет пара (размер-тег). Размер блока в байтах, так можно узнать, например, длину строки. А тег памяти - один из вариантов **box_type**. Зная тип каждого элемента можно рекурсивно обходить дерево разбора. Это тоже довольно популярная техника.

- 3) Построение AST³, мы немного говорили об этом в главе [2.5. Архитектура. Стековые машины vs регистровых](#). Синтаксическое дерево состоит из большого количества небольших элементов (нод), так что это первый кандидат на использование пула памяти.

Ноды устроены однотипно, это массив из элементов размером в указатель, в данном случае 8 байт: Это соответствует структуре **c_tree_t**, в начале которой стоит тег ноды (не путаем с тегом памяти), описывающий что будет дальше, потом данные - union, каждый элемент которого соответствует тегу. Создаются они функцией драйвера пула памяти `box_mp_t::mk_node`.

Например, правило грамматики **some_identifier** описывается так:

```
some_identifier
: IDENTIFIER
{
    $$ = (TT*)driver->mk_node (2,
                                STMT_IDENTIFIER,
                                driver->mp_.box_string ($1));
};
```

Здесь вызывается функция `mk_node`, первый аргумент указывает на размер ноды в 8-байтных элементах, далее идёт тег `STMT_IDENTIFIER` и вторым элементом будет указатель на копию строки, полученной при лексическом разборе. Тегу `STMT_IDENTIFIER` соответствует элемент union'a `identifier_`, который описан так:

```
struct {
    const wchar_t *name;
} identifier_;
```

Как видим, в структуре единственный член - указатель на строку.

Еще пример - операция логического И:

```
logical_and_expression
: inclusive_or_expression
```

³ Abstract Syntax Tree

```

| logical_and_expression AND_OP inclusive_or_expression
{
    $$ = (c_tree_t*)driver->mk_node (3, STMT_BOP_AND, $1, $3);
};

```

В данном случае тег - STMT_BOP_AND соответствует элементу union

```

struct
{
    TT      *left;
    TT      *right;
} bop_;

```

Т.е. в структуре два указателя на аргументы операции + тег.

Еще стоит упомянуть списки, например, у функции может быть произвольное количество аргументов, грамматика описывает это рекурсивно:

```

parameter_list
: parameter_declaration
{
    $$ = driver->mp_cons_set ($1, NULL);
}
| parameter_list ',' parameter_declaration
{
    $$ = driver->mp_box_set_conc ($1, driver->mp_cons_set ($3, NULL));
};

```

Случай пустого списка описан в правиле более высокого уровня, хотя, вполне можно было описать его и здесь третьим вариантом. Функция `driver->mp_cons_set` создаёт элемент списка, в первом случае это список из одного элемента, во втором случае списки объединяются. Таким образом на выходе из правила ***parameter_list*** получим список надлежащей длины. При необходимости список нетрудно преобразовать в массив с помощью функции `driver->mp_list_to_array`.

В конце концов по окончании синтаксического разбора получим вершину AST как результат разбора правила самого верхнего уровня ***translation_unit***.

- 4) Разбор дерева AST с генерацией инструкций ассемблера. Тегированная память позволяет заметно упростить разбор синтаксического дерева. Тег конкретного указателя указывает либо на список, либо на массив, либо на литерал (строка, число,...).

Список (например, список функций, глобальных переменных,...) обходим, рекурсивно вызывая разбор каждого указателя из него.

Массив может оказаться только в случае элемента типа ***c_tree_t***. В этом случае следует продолжить разбор, опираясь на тип/тег этого ***c_tree_t***. Например, в случае тега STMT_BOP_AND, мы понимаем, что это логическое и, которое содержит две ветви, каждая из которых должна закончиться вычислением

логического (целочисленного) значения. Результатом разбора этого элемента дерева должно быть вычисление значения логического И от этих двух значений.

Чтобы этого достичь, требуется рекурсивно обойти каждую из этих ветвей. При обходе будут сгенерированы потоки инструкций, каждая из которых оставит на вершине стека значение. Осталось преобразовать типы к целочисленным (сейчас это не актуально, у нас все вычисления целочисленные, но в дальнейшем непременно потребуется). Затем добавить к получившемуся коду инструкцию логического И, которая удалит со стека старые значения и добавит результат.

Как это делается? Для стековой машины генерация инструкций проста и красива.

- a) Если нам встретилась константа, добавляем инструкцию `imdpush`
 - b) Переменная - `varpushd`
 - c) Lvalue (присвоение) - `varpush` (в отличие от `varpushd` записывает в стек адрес переменной, а не её значение)
 - d) операция - добавляем инструкцию, соответствующую этой операции
 - e) вызов функции - инструкцию `call`
- 5) Набор инструкций описан в файлах [common/instr.*](#). Там есть и мнемоники инструкций и их опкоды (идентификаторы). Инструкции бывают двух типов - из потока управления или потока исполнения. По коду инструкции и типу потока можно получить описатель инструкции (`instr_def_t`). Описатель содержит число и типы параметров инструкции, например, у инструкции `imdpush` один параметр - целое число. Следовательно, при добавлении такой инструкции следует добавить и аргумент.

```
case BT_INT:
{
    new_inst(get_asm_instr(ioCXimdpush, false), code);
    inst_t *pinst = new_inst(NULL, code);
    pinst->ival_ = (int32_t)box_mp_t::unbox_int(tree);
    tpush(ftInteger32);
    break;
}
```

Здесь:

- a) `get_asm_instr(ioCXimdpush, false)` - получаем описатель инструкции
- b) `code` - создаваемый список (поток) инструкций
- c) `new_inst` добавляет новую инструкцию к потоку
- d) `new_inst(NULL, code)` добавляет в поток пустое место под аргумент
- e) `box_mp_t::unbox_int` преобразует тегированный указатель из дерева разбора в целое. Тег `BT_INT` говорит нам что там именно оно.
- f) `tpush(ftInteger32)` - записывает в стек типов тип текущего значения. Для корректного преобразования типов требуется знать типы значений в стеке, с этой целью присутствует стек типов, за которым приходится тщательно следить. При использовании бинарных операций в стеке могут оказаться операнды разных типов (`int` & `double`, например) и нам

необходим механизм автоматического приведения типов. Такой механизм, на основании типов на вершине стека, приведёт аргументы к одному типу и вставит инструкцию операции правильного типа.

В данный момент это не требуется, но в дальнейшем обязательно пригодится.

- 6) После обхода синтаксического дерева имеем список глобальных переменных, список компилируемых функций. Для каждой функции в наличии поток инструкций, который следует преобразовать в вид, пригодный для ассемблера. У нас есть всё, что нужно для этого - мы знаем к какому потоку относится каждая инструкция, требуется разбить на потоки, кое-где, добавить инструкции `exec/stop` для переключения с потока на поток

Эмулятор

Эмулятор процессора - программное средство, предназначенное для симуляции работы процессора. Ранее мы отлаживали работу процессора с помощью эмуляции аппаратной части. На тот момент было неочевидно, что процессор с подобной архитектурой вообще способен работать. Но сейчас, преодолев основные трудности аппаратной реализации, будем считать процессор просто автоматом, работающим согласно спецификации. Спецификацией в данном случае является система команд.

Исходные коды эмулятора можно найти [здесь](#).

Эмулятор принимает файлы, созданные ассемблером [zasm](#): `mem_rom image.mem` (бинарное представление исполняемого кода) и `data_segment.mem` (сегмент глобальных данных). А также начальный адрес исполнения, который можно взять в `.map` файле, подготовленном `zasm`.

Пример

Компилировать станем вот такой текст:

```
int a(int m, int n)
{
    if (0 == m)
        return n + 1;
    if (0 == n)
        return a(m - 1, 1);
    return a(m - 1, a(m, n - 1));
}
int main()
{
    a(3, 8);
    return 0;
}
```

Функция **a**, кстати, несмотря на кажущуюся простоту совершенно нетривиальна. Это т.н. [функция Аккермана](#) (1928).

В главе [4.3. Теория. Вычислимость, машина Тьюринга, ...](#) мы вскользь касались примитивно-рекурсивных функций (ПРФ) - это всё, что можно получить из примитивных функций (0, 1, выбор из списка (*проекция*)) с помощью итераций.

Например, операторы сложения, умножения, возведения в степень являются ПРФ.

Также упоминались частично рекурсивные функции (ЧРФ) - всё, что можно вычислить с помощью λ -исчисления и/или машины Тьюринга.

Так вот, функция Аккермана не примитивно рекурсивна, она относится к ЧРФ, оставаясь при этом определённой для любых своих аргументов (т.е. математическим языком, *тотальна*).

Функция Аккермана - частный случай т.н. [супероператора](#). Чуть выше упоминались три привычные нам математических оператора: сложение, умножение и возведение в степень. С точки зрения математики, заманчиво иметь один универсальный оператор, который в зависимости от своего дополнительного аргумента может работать как любой из упомянутой тройки. Функция Аккермана предоставляет нам частный случай такого оператора. Более общий случай даёт теорема/[последовательность Гудстейна](#), но это совсем другая история.

Итак:

- $a(1, x) \Rightarrow 2 + x$
- $a(2, x) \Rightarrow 2 * x + 3$
- $a(3, x) \Rightarrow 2^{x+3} - 3$
- $a(4, x) \Rightarrow \underbrace{2^{2^{\dots^2}}}_{x+3} - 3$
- $a(5, x) \Rightarrow \underbrace{2^{2^{\dots^2}}}_{\underbrace{2^{2^{\dots^2}}}_{x+3} - 3 \text{ (всего } x \text{ блоков } 2^{2^{\dots^2}})}}$
- ...

Как видим, первые три строчки - примитивные операторы, далее идут т.н. [тетрация](#) и [пентация](#), следом, очевидно, должны идти гексация и гептация, но их свойства настолько не исследованы, что и имена собственные не присвоены.

Удивительным образом, функция Аккермана говорит нам, что существует бесконечное количество возможных операторов, пусть, практическую ценность (в данный момент) имеют только первые три. Таким образом, несколько строк кода, что мы собираемся скомпилировать, скрывают саму суть бесконечной сложности мироздания.

Итого

Собираем компилятор [zz1](#), компилируем выше приведённый текст и получаем

```
// Proc: a
CCODE {
a:
    exec @blk_1 ;
    if @endif_1 ;
@thencode_1:
    exec @blk_2 ;
    ret ;
@endif_1:
    exec @blk_3 ;
    if @endif_2 ;
@thencode_2:
    exec @blk_4 ;
    call a 8 0 ;
    ret ;
@endif_2:
    exec @blk_5 ;
    call a 8 0 ;
    call a 8 0 ;
    ret ;
}

XCODE {
@blk_1:
    locpushd 0 ;
    imdpush 0 ;
    eq ;
    stop ;
}
XCODE {
@blk_2:
    locpushd 4 ;
    imdpush 1 ;
    add ;
    stop ;
}
XCODE {
@blk_3:
    locpushd 4 ;
    imdpush 0 ;
    eq ;
    stop ;
}
XCODE {
@blk_4:
    locpushd 0 ;
    imdpush 1 ;
    sub ;
    imdpush 1 ;
    stop ;
}
XCODE {
@blk_5:
    locpushd 0 ;
    imdpush 1 ;
    sub ;
    locpushd 0 ;
    locpushd 4 ;
    imdpush 1 ;
    sub ;
    stop ;
}
}
```



```
// Proc: main
CCODE {
main:
    exec @blk_6 ;
    call a 8 0 ;
    exec @blk_7 ;
    ret ;
}
XCODE {
@blk_6:
    imdpush 3 ;
    imdpush 10 ;
    stop ;
}
XCODE {
@blk_7:
    pop ;
    imdpush 0 ;
    stop ;
}
```

Пропускаем этот код через ассемблер

```
DATA:
CCODE:
    @endif_1: 0x39
    @endif_2: 0x46
    @thencode_1: 0x36
    @thencode_2: 0x3e
    a: 0x31
    main: 0x54
CCODE:
    @blk_1: 0x0
    @blk_2: 0x6
    @blk_3: 0xc
    @blk_4: 0x12
    @blk_5: 0x1a
    @blk_6: 0x27
    @blk_7: 0x2c
```

и вот карта памяти и собственно код:

```
11 00 09 00 38 00 11 04
09 01 1c 00 11 04 09 00
38 00 11 00 09 01 24 09
01 00 11 00 09 01 24 11
00 11 04 09 01 24 00 09
03 09 08 00 88 01 09 00
00 09 00 11 39 04 09 06
14 09 0c 11 46 04 09 12
0f 31 08 00 04 14 09 1a
0f 31 08 00 04 0f 31 08
00 04 14 00 09 27 0f 31
08 00 04 09 2c 14 00
```

Фиг.5.3.6 код для примера (data_segment.mem)

Задаём стартовый адрес (main: 0x54, определяется как первый аргумент программы эмулятора), запускаем и получаем:

```
exec 27
imdpush 3
imdpush 8
call 31 (3, 8)
exec 0
locpush 3
imdpush 0
eq 0,3 => 0
if (0) 39
exec c
locpush 8
imdpush 0
eq 0,8 => 0
if (0) 46
exec 1a
locpush 3
imdpush 1
sub 1,3 => 2
locpush 3
locpush 8
imdpush 1
sub 1,8 => 7
call 31 (3, 7)
...
add 1,2044 => 2045
ret 2045
stop
ret 2045
stop
ret 2045
stop
ret 2045
stop
exec 2c
pop
imdpush 0
ret 0
```

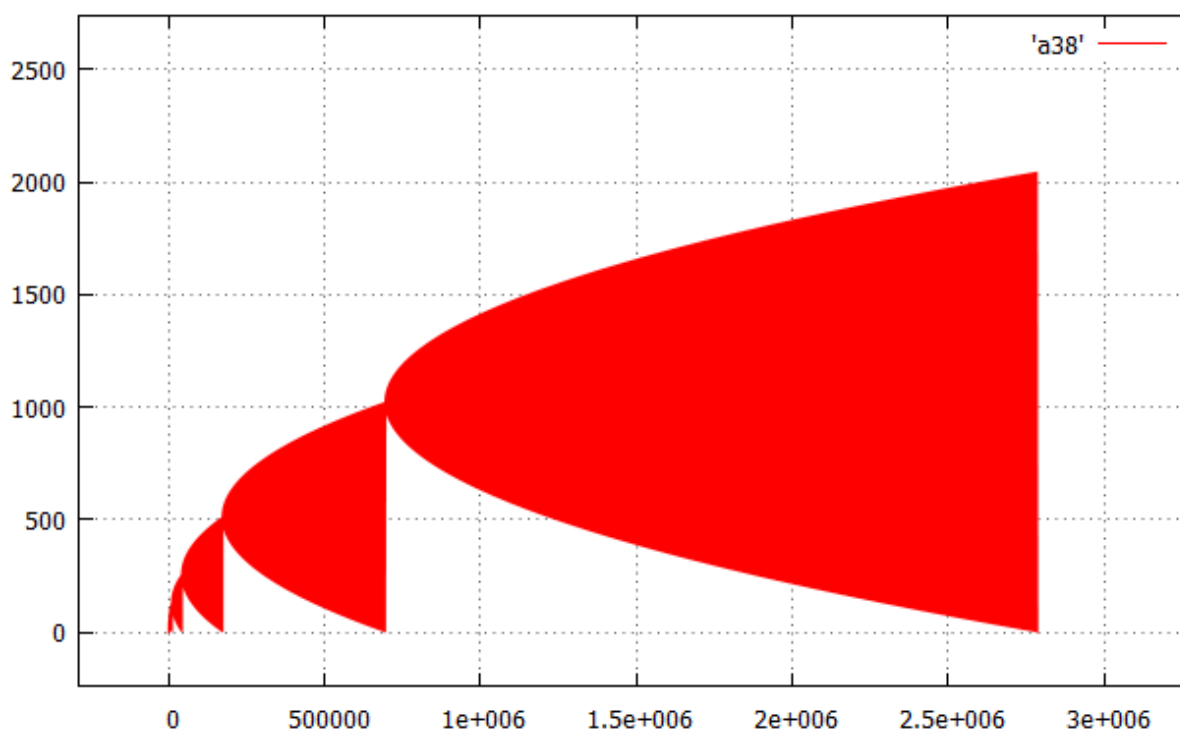
Итак, полученный результат равен **2045**.

Сверимся с предсказанием. $a(3, x) \Rightarrow 2^{x+3} - 3$

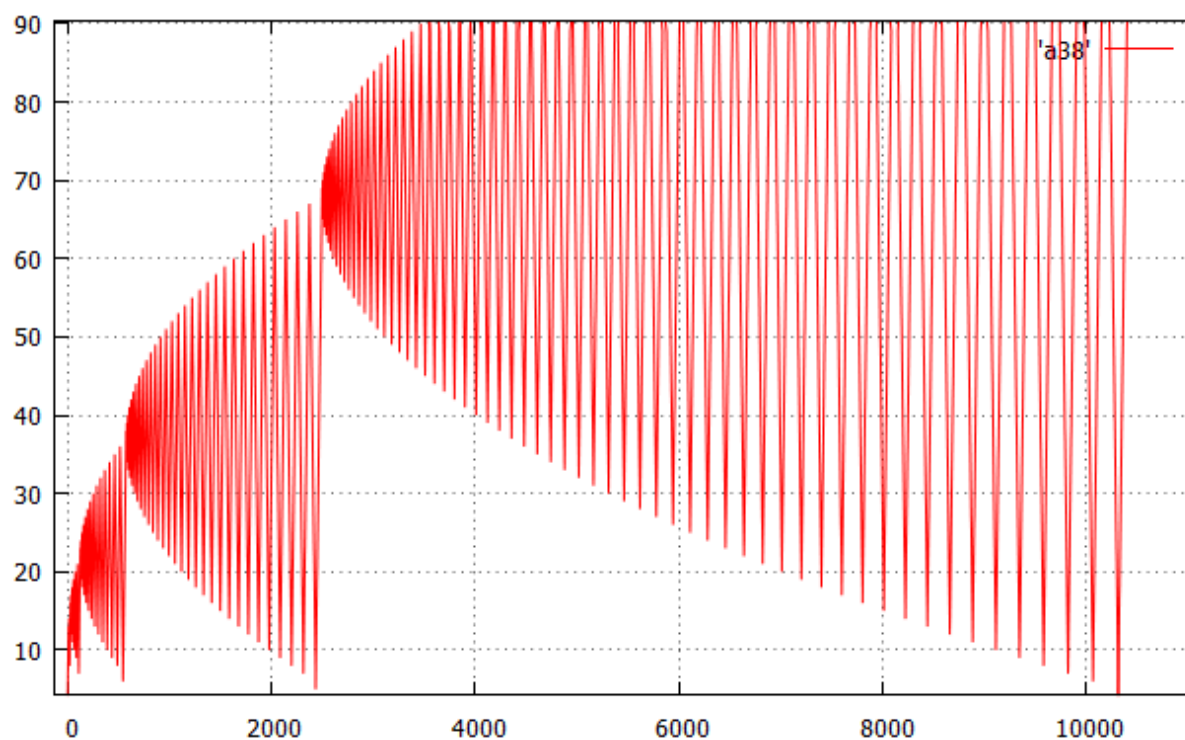
для $x=8$ ответ должен быть $2^{8+3} - 3 \Rightarrow 2^{11} - 3 \Rightarrow 2048 - 3 \Rightarrow \mathbf{2045}$

Сходится.

Небольшой бонус (в эмуляторе можно позволить себе всё что угодно):



Фиг.5.3.7 Глубина рекурсии функции $a(3,8)$ (Y) в процессе исполнения (Y-время)



Фиг.5.3.8 То же более подробно